

```
class NSAMPFeedbackLoops(nn.Module):
    def __init__(self, N=128, max_iter=5):
        super().__init__()
        self.N = N
        self.max_iter = max_iter

        # Хопфилдовские веса
        self.W1 = nn.Parameter(torch.randn(N,
N))
        self.W2 = nn.Parameter(torch.randn(N,
N))

        # Эталонная память (один паттерн x0)
        self.x0 = nn.Parameter(torch.randn(N))

    def single_iteration(self, xin):
        """Одна итерация внутренней петли"""
        # Блок 2: Двухслойная ИНС
        h1 = torch.tanh(xin @ self.W1)
        h2 = torch.tanh(h1 @ self.W2)
        xout = h2
```

Блок 3-4: Сравнение

```
success = (xout == self.x0).float().mean()
```

```
distance = F.mse_loss(xout, self.x0)
```

```
return xout, success, distance
```

```
def internal_feedback_loop(self, xin):
```

```
    """Внутренняя петля 1-2-3-4-1"""
```

```
    x_current = xin
```

```
    trajectory = [x_current]
```

```
    for iter in range(self.max_iter):
```

```
        xout, success, dist =
```

```
self.single_iteration(x_current)
```

```
        trajectory.append(xout)
```

```
    if success > 0.95: # Успех
```

```
        return {
```

```
            'success': True,
```

```
            'iterations': iter+1,
```

```
            'trajectory': trajectory,
```

```
            'final_output': xout,
```

```
        'distance': dist
    }
```

```
x_current = xout # Обратная связь
```

```
# Неудача → чувство знания
```

```
metamemory_strength = 1.0 - dist.item()
```

```
tip_of_tongue = metamemory_strength <
```

```
0.3
```

```
return {
```

```
    'success': False,
```

```
    'iterations': self.max_iter,
```

```
    'trajectory': trajectory,
```

```
    'final_output': xout,
```

```
    'distance': dist,
```

```
    'tip_of_tongue': tip_of_tongue,
```

```
    'metamemory_strength':
```

```
metamemory_strength
```

```
}
```